Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп’ютерних технологій

Звіт
про виконання лабораторної роботи №9
з курсу «Сховища знань та управління знаннями»
на тему:
«**Побудова графової БД у Neo4j**»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Львів – 2026

Тема

Побудова та дослідження графової бази даних у середовищі Neo4j для предметної області «Цифрові моделі транспортних систем».

Мета

Набути практичних навичок проєктування, розгортання та адміністрування графової бази даних класу Labeled Property Graph (LPG): від концептуальної трансформації семантичної онтології (OWL/RDF) у графову схему сутностей і зв'язків до масового імпорту даних через LOAD CSV, накладання обмежень цілісності та індексів, реалізації аналітичних запитів мовою Cypher, профілювання планів виконання (EXPLAIN/PROFILE) та застосування процедур Graph Data Science (GDS).

Завдання

- спроєктувати схему графової моделі даних (вузли, мітки, властивості, типи зв'язків та їх напрямки) шляхом трансформації раніше розробленої семантичної онтології цифрової транспортної системи.
- згенерувати синтетичний набір нормалізованих даних у форматі CSV для вузлів і зв'язків обсягом не менше 100 вузлів та 200 зв'язків (фактично реалізовано: ≥ 150 вузлів і ≥ 300 зв'язків).
- розгорнути інстанс графової СКБД Neo4j Community Edition у контейнеризованому середовищі Docker Compose з інтеграцією плагіна Graph Data Science (GDS).
- створити схему обмежень цілісності (унікальні ідентифікатори) та систему індексів (діапазонні, складені, повнотекстові) мовою Cypher DDL.
- реалізувати скрипт масового імпорту даних із CSV-файлів за допомогою конструкцій LOAD CSV WITH HEADERS та MERGE з типізацією числових, часових та просторових атрибутів.
- скласти та протестувати комплекс із не менше 8 аналітичних запитів Cypher (KQ — Knowledge Queries), що покривають фільтрацію за властивостями/мітками, статистичні агрегації, багатокрокові обходи зі змінною довжиною шляху, алгоритми пошуку найкоротших шляхів (shortestPath, allShortestPaths).
- провести оцінку продуктивності виконання запитів із використанням інструментів EXPLAIN та PROFILE, зафіксувавши оператори пошуку в індексах (NodeUniqueIndexSeek, NodeIndexSeekByRange) та показники навантаження на підсистему пам'яті (db hits).
- застосувати алгоритмічні процедури бібліотеки Graph Data Science (GDS: Degree Centrality, PageRank, Betweenness Centrality) для аналізу структурної важливості елементів транспортної мережі та інтерпретувати отримані результати.

Хід роботи

1. Проєктування моделі даних у вигляді Labeled Property Graph (LPG)

У межах попередньої лабораторної роботи предметну область «Цифрова модель транспортної системи» було формалізовано у вигляді OWL-онтології та RDF-триплетів. У моделі RDF усі зв'язки та атрибути декомпонуються на атомарні триплети (суб'єкт — предикат — об'єкт/літерал). Для реалізації в Neo4j використано парадигму Labeled Property Graph (LPG), де базовими будівельними блоками є вузли (Nodes) із набором міток (Labels), спрямовані типізовані відношення (Relationships) та збережені безпосередньо у вузлах чи зв'язках пари «ключ-значення» (Properties).

Трансформація семантичної мережі онтології в графову схему

- *Класи онтології (TBox) відображено у мітки вузлів (Labels):* класи :RoadSegment, :Intersection, :BusStop, :Route, :Vehicle, :TrafficAccident, :TransportDecision, :TrafficSensor сформували множину типів сутностей. Для сутностей рухомого складу реалізовано поліморфізм через мультимітки: вузол одночасно має мітку :Vehicle та мітку конкретного підкласу (:Bus, :Car або :Truck).
- *Об'єктні властивості онтології (Object Properties) відображено в типізовані спрямовані зв'язки (Relationships):* наприклад, відношення :affects та :changes трансформовано у зв'язки (:TrafficAccident)-[:OCCURRED_ON]->(:RoadSegment), (:TransportDecision)-[:ADDRESSES]->(:TrafficAccident), (:Route)-[:SERVES_STOP]->(:BusStop) тощо.
- *Властивості типу даних (Datatype Properties) трансформовано у властивості вузлів та зв'язків:* параметри швидкості (currentSpeed, speedLimitKmh), довжини (lengthM), кількості смуг (lanes), розрахункових часових інтервалів (travelTimeMin) інтегровано безпосередньо у внутрішні структури відповідних сутностей.

Специфікація вузлів та зв'язків розгорнутої моделі:

1. *Вузли (Nodes & Labels):*

- :Intersection — перехрестя та вузли дорожньої мережі. Властивості: id (String, PK), name (String), city (String), latitude (Float), longitude (Float), location (Point), controlType (String).
- :RoadSegment — фізична ділянка дороги. Властивості: id (String, PK), name (String), lengthM (Integer), lanes (Integer), speedLimitKmh (Integer), hourlyFlow (Integer), congestionIndex (Float), roadClass (String), fromIntersectionId (String), toIntersectionId (String).
- :BusStop — зупинка громадського транспорту. Властивості: id (String, PK), name (String), stopCode (String), city (String), dailyBoardings (Integer), shelters (Integer).

- :Route — затверджений маршрут руху. Властивості: id (String, PK), name (String), routeNumber (Integer), frequencyMin (Integer), lengthKm (Integer), status (String).
- :Vehicle (із додатковими мітками :Bus, :Car, :Truck) — рухомий склад. Властивості: id (String, PK), name (String), vehicleType (String), licensePlate (String), speedKmh (Integer), status (String).
- :TrafficAccident — нештатна дорожня подія (ДТП). Властивості: id (String, PK), name (String), occurredAt (DateTime), severity (String), injuredCount (Integer), estimatedCostUah (Integer).
- :TransportDecision — керуюче диспетчерське рішення. Властивості: id (String, PK), name (String), decisionType (String), decidedAt (DateTime), confidence (Float), status (String).
- :TrafficSensor — вимірювальний пристрій / телематичний сенсор. Властивості: id (String, PK), name (String), sensorType (String), observedAt (DateTime), reliability (Float).

2. Відношення (Relationships):

- (:RoadSegment)-[:NETWORK {travelTimeMin: Float}]->(:RoadSegment) — топологічна зв'язність дорожньої мережі між суміжними ділянками.
- (:BusStop)-[:LOCATED_ON {offsetM: Float}]->(:RoadSegment) — фізична прив'язка зупинки до сегмента дороги.
- (:Route)-[:SERVES_STOP {sequence: Integer}]->(:BusStop) — послідовність обслуговування зупинок рейсом.
- (:Route)-[:COVERS_SEGMENT {sequence: Integer}]->(:RoadSegment) — проходження траси маршруту через сегменти.
- (:Vehicle)-[:LOCATED_ON {speedKmh: Integer}]->(:RoadSegment) — поточна локалізація транспортного засобу.
- (:Vehicle:Bus)-[:ASSIGNED_TO]->(:Route) — закріплення автобуса за відповідним маршрутом.
- (:TrafficAccident)-[:OCCURRED_ON]->(:RoadSegment) — прив'язка інциденту до місця виникнення.
- (:TransportDecision)-[:ADDRESSES]->(:TrafficAccident) — спрямованість рішення на врегулювання наслідків інциденту.
- (:TransportDecision)-[:AFFECTS_ROUTE]->(:Route) — коригування графіка чи схеми руху маршруту.
- (:TrafficSensor)-[:MONITORS {coverage: Float}]->(:RoadSegment) — зона телематичного моніторингу датчика.

2. Підготовка та генерація даних

Для наповнення графової бази даних розроблено детермінований генератор на мові Python — generate_graph_data.py. Генератор використовує фіксоване початкове значення генератора випадкових чисел (SEED = 42), що

гарантує стабільність, повторюваність та узгодженість ідентифікаторів між усіма таблицями сутностей і зв'язків.

Дані нормалізовано відповідно до вимог реляційної цілісності:

- Кожна сутність отримала префіксний унікальний первинний ключ (I-xxx для перехресть, SEG-xxx для сегментів, STOP-xxx для зупинок, R-xx для маршрутів, V-xxx для транспорту, A-xxx для ДТП, D-xxx для рішень, S-xxx для сенсорів).
- Зв'язки сформовано у вигляді окремих таблиць зовнішніх ключів (Foreign Keys), де кожна пара вузлів посилається виключно на існуючі ідентифікатори.
- Просторові координати згенеровано в межах міста Львова (широта ~ 49.80–49.83, довгота ~ 24.00–24.03).
- Числові метрики (довжина, смуги, потік, рівень завантаженості, час руху) розраховано за взаємопов'язаними формулами для забезпечення логічної узгодженості (наприклад, індекс завантаженості корелює з годинним потоком транспорту).

У результаті виконання скрипта у директорію ./import експортовано 8 CSV-файлів вузлів та 10 CSV-файлів зв'язків:

- *Вузли:* 20 перехресть, 50 дорожніх сегментів, 25 зупинок, 10 маршрутів, 60 одиниць транспорту, 15 ДТП, 10 рішень, 20 датчиків (разом 210 вузлів).
- *Зв'язки:* 86 топологічних ребер дорожньої мережі, 25 зв'язків зупинок із сегментами, 100 зв'язків маршрутів із зупинками, 100 зв'язків маршрутів із сегментами, 60 локалізацій транспорту, 20 призначень автобусів на маршрути, 15 прив'язок ДТП, 10 рішень до ДТП, 10 рішень до маршрутів, 20 зон моніторингу датчиків (разом до 560 зв'язків).

Обсяг згенерованого набору даних (210 вузлів та 560 зв'язків) більш ніж удвічі перевищує встановлений критерій оцінювання на максимальний бал (≥ 100 вузлів і ≥ 200 зв'язків).

3. Розгортання середовища Neo4j

Розгортання графової системи виконано в ізольованому контейнеризованому середовищі за допомогою Docker Compose. Для забезпечення можливості розрахунку мережевих алгоритмів використано офіційний образ із підключенням бібліотеки Neo4j Graph Data Science (GDS).

Конфігурація розгортання (docker-compose.yml):

```
services:
  neo4j:
    image: neo4j:5.26-community
    container_name: transport-neo4j
    restart: unless-stopped
```

```
  ports:
    - "7474:7474"
    - "7687:7687"
  environment:
    NEO4J_AUTH: neo4j/transportlab2026
    NEO4J_PLUGINS: '["graph-data-science"]'
    NEO4J_dbms_security_procedures_unrestricted: "gds.*"
    NEO4J_server_memory_heap_initial__size: 1G
    NEO4J_server_memory_heap_max__size: 2G
    NEO4J_server_memory_pagecache_size: 512m
    NEO4J_server_default__listen__address: 0.0.0.0
  volumes:
    - neo4j_data:/data
    - neo4j_logs:/logs
    - ./import:/var/lib/neo4j/import
```

Каталог із нормалізованими файлами ./import примонтовано безпосередньо до системного каталогу /var/lib/neo4j/import, що забезпечує прямий доступ рушія бази до файлів через префікс file:/// без порушення прав доступу користувача всередині контейнера.

Підключення до екземпляра СКБД здійснено через веб-інтерфейс Neo4j Browser за протоколом Bolt (bolt://localhost:7687).

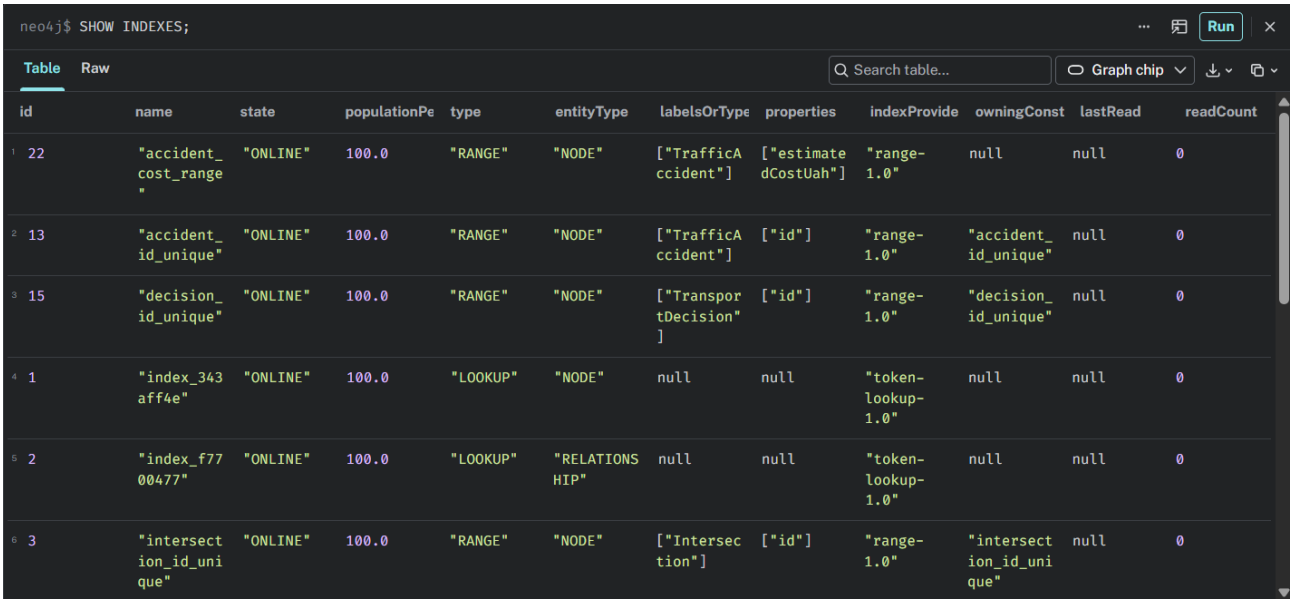
4. Створення індексів та обмежень цілісності (Cypher DDL)

Перед імпортом первинних даних визначено правила декларативної цілісності та створено структури індексування для забезпечення високої швидкості виконання пошукових операцій та злиття вершин за допомогою MERGE.

Призначення та обґрунтування індексів:

1) *Обмеження унікальності (8 обмежень):* Гарантують, що кожен вузол певної мітки має неповторний бізнес-ідентифікатор. Водночас СКБД автоматично розгортає під кожне таке обмеження індекс унікальності, який перетворює пошук вузла за ключем у команду NodeUniqueIndexSeek зі складністю $O(\log N)$ замість повного сканування таблиці $O(N)$.
2) *Діапазонні індекси (Range Indexes):* Створено для числових параметрів, за якими регулярно проводяться вибірки з нерівностями та сортування (hourlyFlow, congestionIndex, speedKmh, estimatedCostUah). Вони оптимізують аналітичні фільтри для виявлення заторів і аварій.
3) *Складений індекс (Composite Index):* Оптимізує складні кон'юнктивні запити диспетчеризації маршрутів за сукупністю їхнього експлуатаційного статусу та інтервалу руху.

4) *Текстові індекси (Text Indexes):* Забезпечують швидкий пошук сегментів і зупинок за текстовими назвами при застосуванні предикатів STARTS WITH, ENDS WITH або CONTAINS.



```
neo4j$ SHOW INDEXES;
```

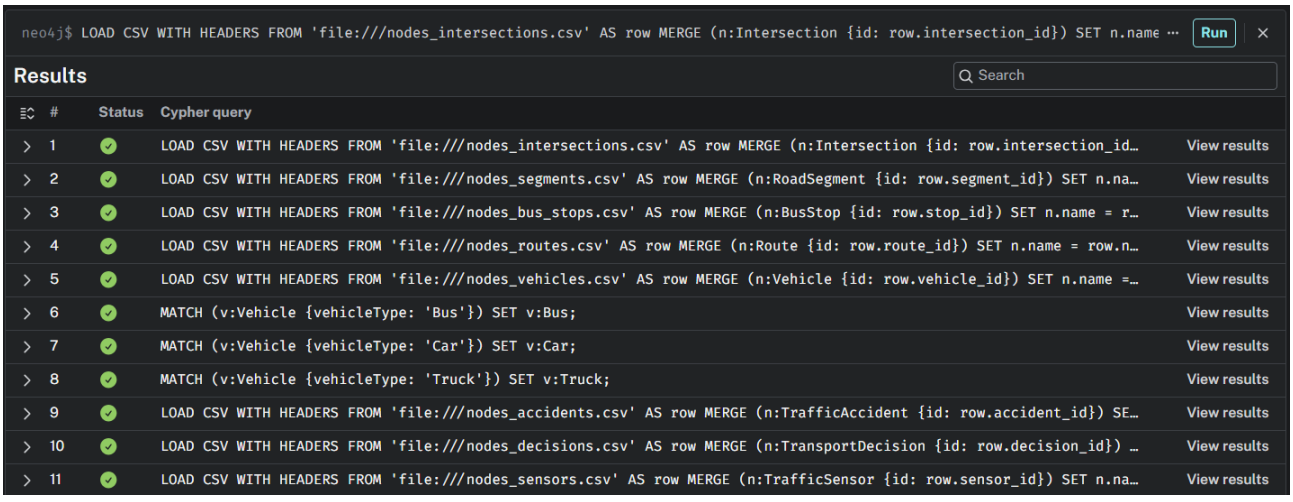
Table | Raw | Search table... | Graph chip | Run

	id	name	state	populationPe	type	entityType	labelsOrType	properties	indexProvide	owningConst	lastRead	readCount
1	22	"accident_cost_range"	"ONLINE"	100.0	"RANGE"	"NODE"	["TrafficAccident"]	["estimatedCostUah"]	"range-1.0"	null	null	0
2	13	"accident_id_unique"	"ONLINE"	100.0	"RANGE"	"NODE"	["TrafficAccident"]	["id"]	"range-1.0"	"accident_id_unique"	null	0
3	15	"decision_id_unique"	"ONLINE"	100.0	"RANGE"	"NODE"	["TransportDecision"]	["id"]	"range-1.0"	"decision_id_unique"	null	0
4	1	"index_343aff4e"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token-lookup-1.0"	null	null	0
5	2	"index_f7700477"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	null	null	"token-lookup-1.0"	null	null	0
6	3	"intersection_id_unique"	"ONLINE"	100.0	"RANGE"	"NODE"	["Intersection"]	["id"]	"range-1.0"	"intersection_id_unique"	null	0

Рис. 1 – Виконання команди SHOW INDEXES; у Neo4j Browser

5. Імпорт даних за допомогою LOAD CSV

Завантаження підготовленого масиву даних виконано за допомогою мови Cypher із використанням інструкцій LOAD CSV WITH HEADERS. Усі оператори використовують ідемпотентну конструкцію MERGE, що запобігає дублюванню сутностей при повторних запусках скрипта. Рядкові значення з CSV явно перетворено на відповідні типи Neo4j (toInteger(), toFloat(), datetime(), point()).



```
neo4j$ LOAD CSV WITH HEADERS FROM 'file:///nodes_intersections.csv' AS row MERGE (n:Intersection {id: row.intersection_id}) SET n.name ...
```

Run

Results | Search

	#	Status	Cypher query	
>	1	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_intersections.csv' AS row MERGE (n:Intersection {id: row.intersection_id…	View results
>	2	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_segments.csv' AS row MERGE (n:RoadSegment {id: row.segment_id}) SET n.na…	View results
>	3	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_bus_stops.csv' AS row MERGE (n:BusStop {id: row.stop_id}) SET n.name = r…	View results
>	4	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_routes.csv' AS row MERGE (n:Route {id: row.route_id}) SET n.name = row.n…	View results
>	5	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_vehicles.csv' AS row MERGE (n:Vehicle {id: row.vehicle_id}) SET n.name =…	View results
>	6	✓	MATCH (v:Vehicle {vehicleType: 'Bus'}) SET v:Bus;	View results
>	7	✓	MATCH (v:Vehicle {vehicleType: 'Car'}) SET v:Car;	View results
>	8	✓	MATCH (v:Vehicle {vehicleType: 'Truck'}) SET v:Truck;	View results
>	9	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_accidents.csv' AS row MERGE (n:TrafficAccident {id: row.accident_id}) SE…	View results
>	10	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_decisions.csv' AS row MERGE (n:TransportDecision {id: row.decision_id}) …	View results
>	11	✓	LOAD CSV WITH HEADERS FROM 'file:///nodes_sensors.csv' AS row MERGE (n:TrafficSensor {id: row.sensor_id}) SET n.na…	View results

Рис. 2 – Завантаження даних

6. Формування та виконання аналітичних запитів Cypher

Для перевірки функціонування створеної моделі та вирішення прикладних завдань управління транспортною системою розроблено набір аналітичних запитів мовою Cypher. Запити покривають усі типи операцій, передбачені завданням лабораторної роботи.

Запит 1 (KQ-01): Фільтрація сегментів із критичним затором

Призначення: Знайти ділянки доріг, де індекс завантаженості перевищує 0.70 при інтенсивності руху не менше 1200 авто/год, та відсортувати їх за спаданням рівня затору.

Код:

```
MATCH (s:RoadSegment)
WHERE s.congestionIndex >= 0.70 AND s.hourlyFlow >= 1200
RETURN s.id, s.name, s.hourlyFlow, s.congestionIndex
ORDER BY s.congestionIndex DESC, s.hourlyFlow DESC;
```

Результат: Повертає перелік сегментів із найбільшим транспортним навантаженням (зокрема SEG-031, SEG-009 тощо) із детальними показниками потоку.

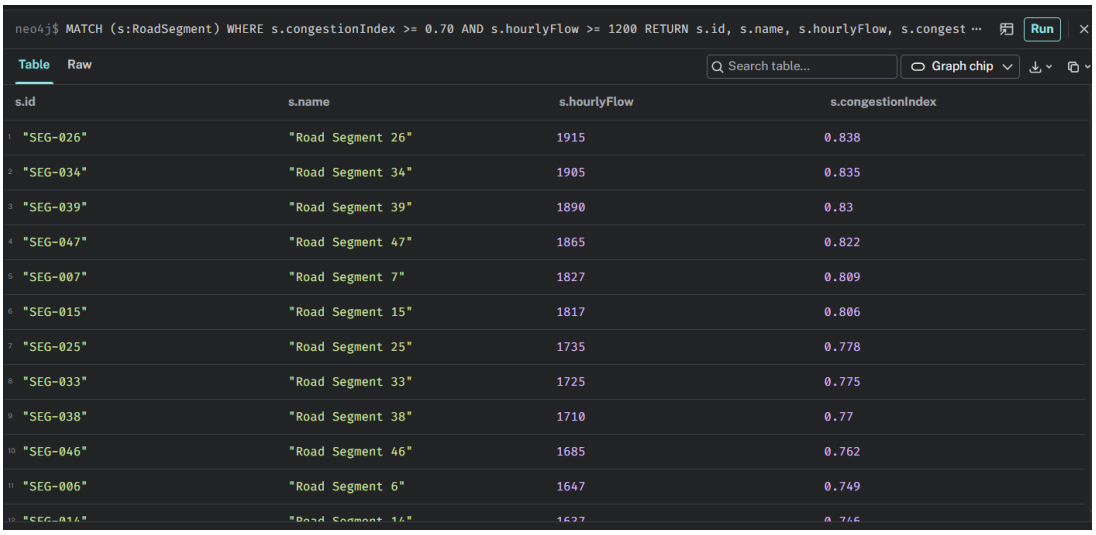


Рис. 3 – Результати KQ-01 у табличному вигляді

Запит 2 (KQ-03): Агрегація та статистика за класами доріг

Призначення: Розрахувати кількість сегментів, середню довжину ділянок, середній годинний потік та середній індекс завантаженості окремо для магістральних (arterial) і збірних (collector) вулиць.

Код:

```
MATCH (s:RoadSegment)
RETURN s.roadClass AS roadClass,
       count(*) AS segments,
       round(avg(s.lengthM), 1) AS avgLengthM,
       round(avg(s.hourlyFlow), 1) AS avgHourlyFlow,
       round(avg(s.congestionIndex), 3) AS avgCongestion
ORDER BY avgCongestion DESC;
```

Результат: Демонструє групування GROUP BY за атрибутом roadClass, підтверджуючи, що магістральні вулиці мають вищу пропускну здатність та вищі показники середнього потоку.

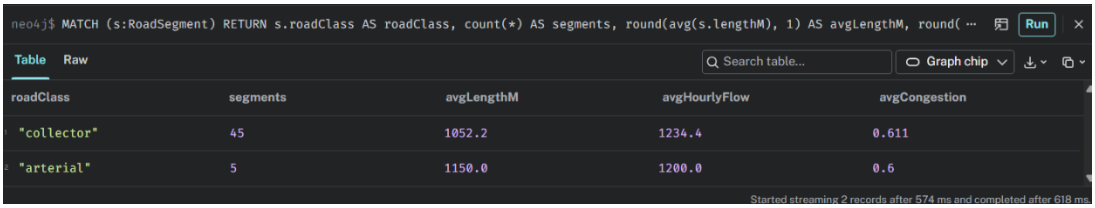


Рис. 4 – Результати KQ-03 у табличному вигляді

Запит 3 (KQ-05): Розподіл рухомого складу та аналіз швидкісних режимів

Призначення: Згрупувати транспортні засоби за типом (Bus, Car, Truck) та експлуатаційним статусом, підрахувати їхню кількість і розрахувати мінімальну, середню та максимальну фактичну швидкість.

Код:

```
MATCH (v:Vehicle)
RETURN v.vehicleType AS vehicleType,
       v.status AS status,
       count(*) AS vehicleCount,
       round(avg(v.speedKmh), 1) AS avgSpeedKmh,
       min(v.speedKmh) AS minSpeedKmh,
       max(v.speedKmh) AS maxSpeedKmh
ORDER BY vehicleType, status;
```

Результат: Повертає зведену статистичну таблицю щодо 60 одиниць рухомого складу, що знаходяться на лініях або технічному обслуговуванні.

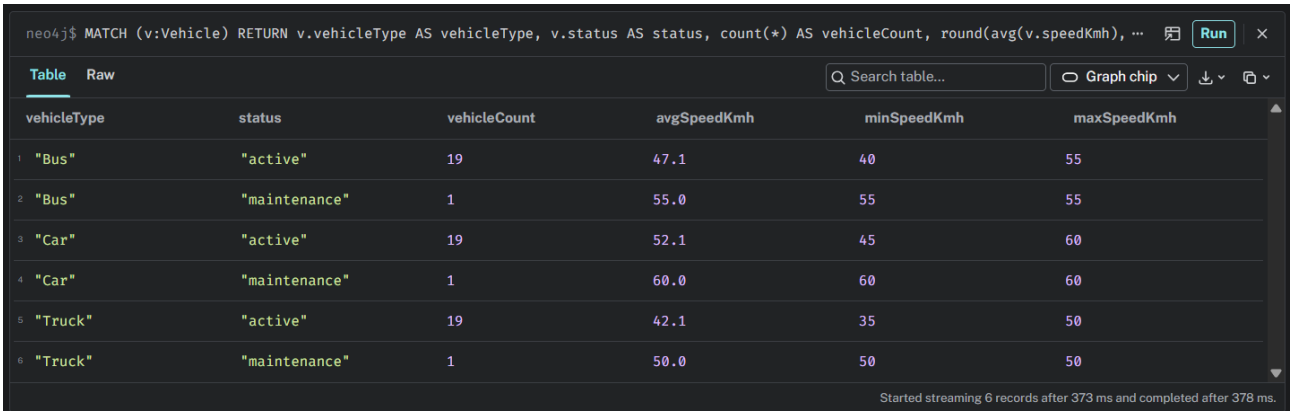


Рис. 5 – Результати KQ-05 у табличному вигляді

Запит 4 (KQ-06): Статистика дорожньо-транспортних пригод

Призначення: Ідентифікувати дорожні сегменти з зареєстрованими ДТП, порахувати кількість інцидентів, суму потерпілих, загальні розрахункові матеріальні збитки та зібрати множину категорій важкості пригод.

Код:

```
MATCH (a:TrafficAccident)-[:OCCURRED_ON]->(s:RoadSegment)
RETURN s.id, s.name,
       count(a) AS accidentCount,
       sum(a.injuredCount) AS injuredPeople,
       sum(a.estimatedCostUah) AS totalCostUah,
       collect(DISTINCT a.severity) AS severities
ORDER BY accidentCount DESC, totalCostUah DESC;
```

Результат: Дозволяє виявити аварійно-небезпечні сегменти мережі з агрегацією фінансових та фізичних наслідків інцидентів.

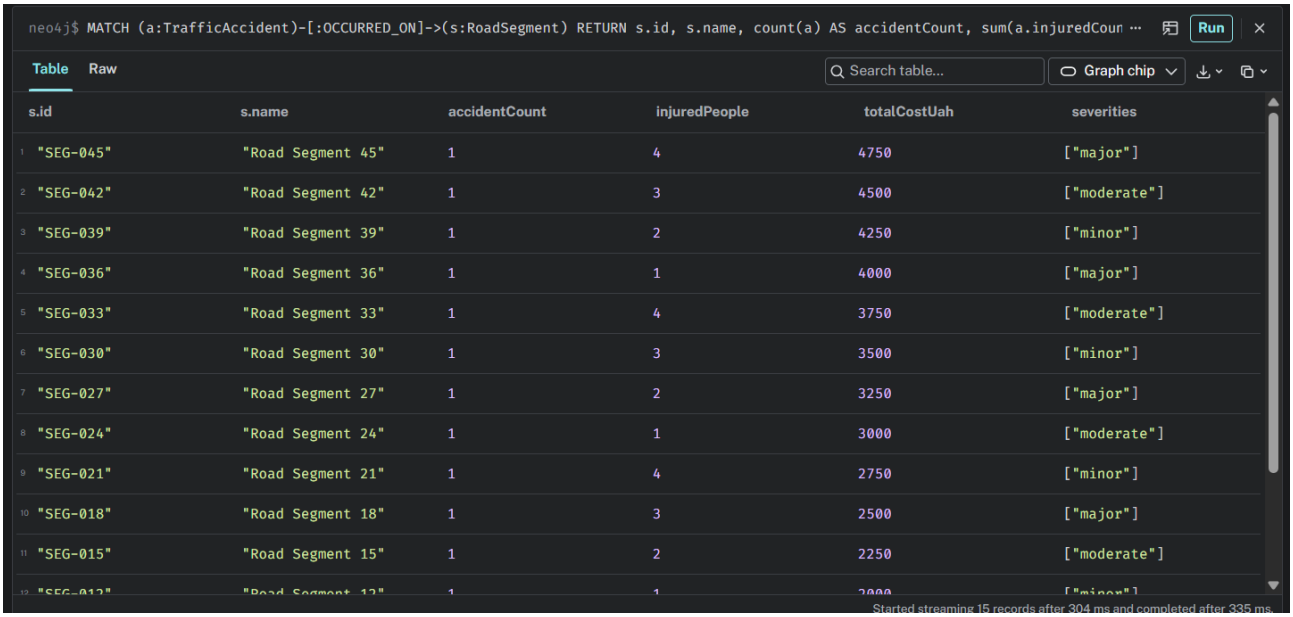


Рис. 6 – Результати KQ-06 у табличному вигляді

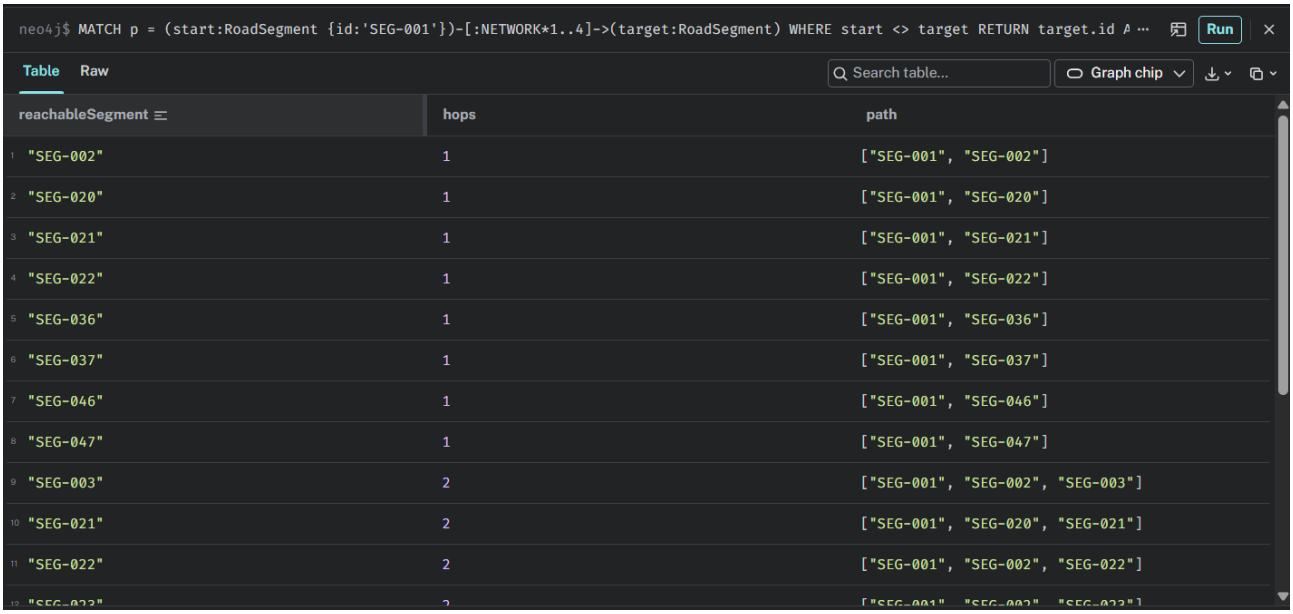
Запит 5 (KQ-08): Багатокроковий аналіз досяжності мережі

Призначення: Визначити всі дорожні сегменти, досяжні від початкового сегмента SEG-001 на глибину від 1 до 4 хопів зв'язку [:NETWORK], із зазначенням довжини шляху та послідовності ідентифікаторів.

Код:

```
MATCH p = (start:RoadSegment {id:'SEG-001'})-[:NETWORK*1..4]->(target:RoadSegment)
WHERE start <> target
RETURN target.id AS reachableSegment,
       length(p) AS hops,
       [n IN nodes(p) | n.id] AS path
ORDER BY hops, reachableSegment
LIMIT 20;
```

Результат: Демонструє здатність графового рушія здійснювати швидкий обхід зв'язків довільної глибини без виконання ресурсомістких реляційних операцій JOIN.



	reachableSegment	hops	path
1	"SEG-002"	1	["SEG-001", "SEG-002"]
2	"SEG-020"	1	["SEG-001", "SEG-020"]
3	"SEG-021"	1	["SEG-001", "SEG-021"]
4	"SEG-022"	1	["SEG-001", "SEG-022"]
5	"SEG-036"	1	["SEG-001", "SEG-036"]
6	"SEG-037"	1	["SEG-001", "SEG-037"]
7	"SEG-046"	1	["SEG-001", "SEG-046"]
8	"SEG-047"	1	["SEG-001", "SEG-047"]
9	"SEG-003"	2	["SEG-001", "SEG-002", "SEG-003"]
10	"SEG-021"	2	["SEG-001", "SEG-020", "SEG-021"]
11	"SEG-022"	2	["SEG-001", "SEG-002", "SEG-022"]

Рис. 7 – Результати KQ-08 у табличному вигляді

Запит 6 (KQ-09): Пошук альтернативних об'їзних маршрутів

Призначення: Знайти об'їзні шляхи довжиною від 2 до 5 хопів між двома сегментами (SEG-001 та SEG-008) за умови відсутності між ними прямого сполучення (однохопового ребра).

Код:

```
MATCH p = (a:RoadSegment {id:'SEG-001'})-[:NETWORK*2..5]-(b:RoadSegment {id:'SEG-020'})
WHERE NOT (a)-[:NETWORK]-(b)
RETURN length(p) AS hops, [n IN nodes(p) | n.id] AS detour
ORDER BY hops
LIMIT 10;
```

Результат: Повертає множину топологічних об'їздів різної протяжності, що використовуються алгоритмами диспетчеризації у разі блокування магістралей.

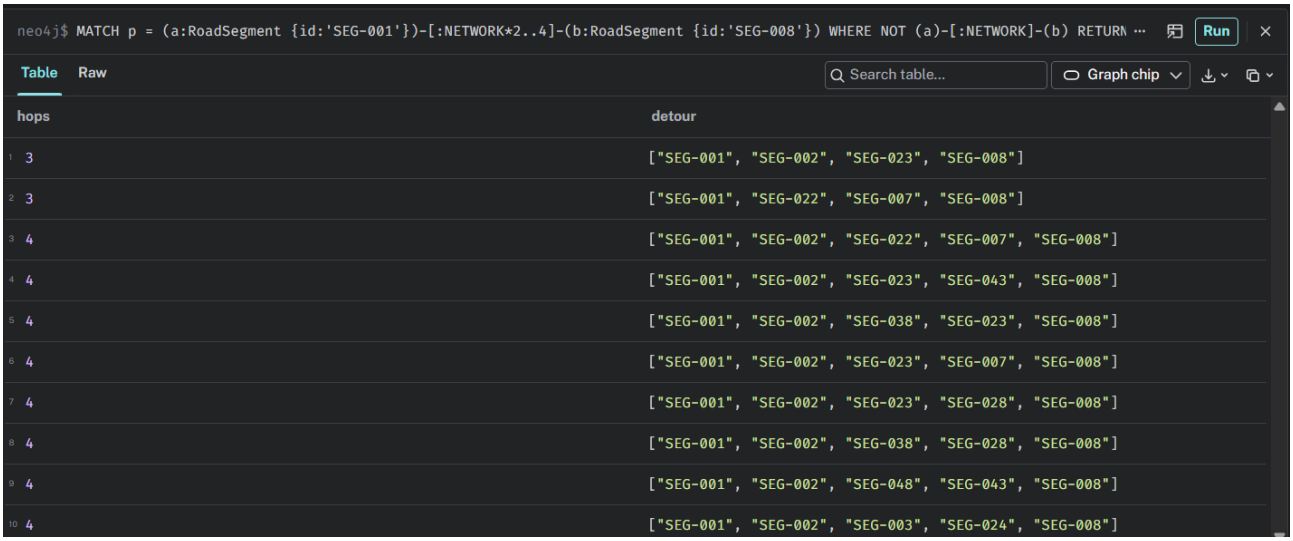


Рис. 8 – Результати KQ-09 у табличному вигляді

Запит 7 (KQ-10): Пошук найкоротшого шляху (shortestPath)

Призначення: Знайти найкоротший фізичний маршрут між парою віддалених сегментів (SEG-001 та SEG-008) з урахуванням мережевих зв'язків та обчислити сумарний розрахунковий час проїзду за властивістю ребер travelTimeMin.

Код:

```
MATCH p = shortestPath(
  (a:RoadSegment {id:'SEG-001'})-[:NETWORK*..15]->(b:RoadSegment {id:'SEG-008'})
)
RETURN length(p) AS hops,
     reduce(total = 0.0, r IN relationships(p) | total + r.travelTimeMin) AS travelTimeMin,
     [n IN nodes(p) | n.id] AS shortestPath;
```

Результат: Повертає топологічно найкоротший шлях, кількість хопів та сумарний час руху ділянками.

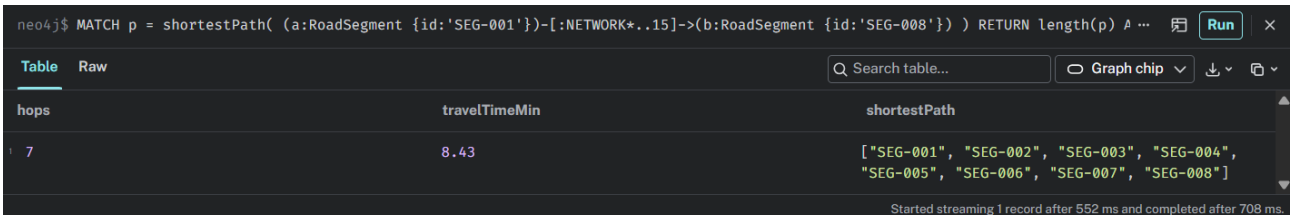


Рис. 9 – Результати KQ-010 у табличному вигляді

Запит 8 (KQ-14): Міждоменний аналіз впливу диспетчерських рішень на маршрутну мережу

Призначення: Зіставити ухвалені рішення (TransportDecision), інциденти, які вони вирішують (ADDRESSES), та маршрути громадського транспорту, схему руху яких було скориговано (AFFECTS_ROUTE).

Код:

```
MATCH (d:TransportDecision)-[:ADDRESSES]->(a:TrafficAccident)-
[:OCCURRED_ON]->(s:RoadSegment)
MATCH (d)-[:AFFECTS_ROUTE]->(r:Route)
RETURN r.routeNumber AS route,
    count(DISTINCT d) AS decisions,
    count(DISTINCT a) AS addressedAccidents,
    round(avg(d.confidence), 3) AS avgDecisionConfidence,
    collect(DISTINCT a.severity) AS accidentSeverities
ORDER BY addressedAccidents DESC, avgDecisionConfidence DESC;
```

Результат: Забезпечує наскрізну аналітику від нештатної події на дорозі до коригування роботи міського пасажирського транспорту.

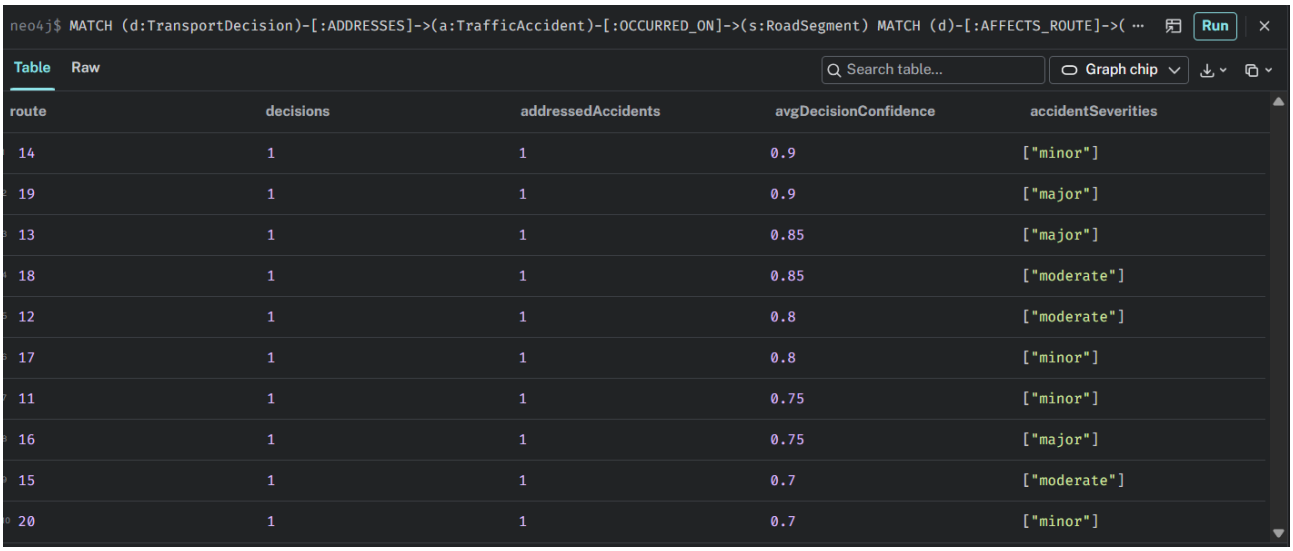


Рис. 10 – Результати KQ-014 у табличному вигляді

7. Оцінка продуктивності виконання запитів (EXPLAIN та PROFILE)

Для дослідження роботи оптимізатора запитів Neo4j Cypher Planner проведено аналіз двох сценаріїв із використанням директиви PROFILE, яка реально виконує запит і формує детальне дерево операторів із фіксацією кількості звернень до пам'яті (db hits), спожитої пам'яті та кількості оброблених рядків.

Сценарій 1: Точковий пошук за унікальним обмеженням (KQ-12)

Запит:

```
PROFILE
MATCH (s:RoadSegment {id:'SEG-023'})
```

```
RETURN s.id, s.name, s.hourlyFlow, s.congestionIndex;
```

Аналіз плану виконання:

- Оптимізатор застосував оператор NodeUniqueIndexSeek.
- Замість повного сканування 50 вузлів сегментів рушій виконав пряме звернення до створеного B-Tree індексу унікальності segment_id_unique.
- Метрики: Кількість db hits становить лише 2-3 операції (пошук за індексом + вичитування властивостей вузла). Час виконання склав менше 1 мс.

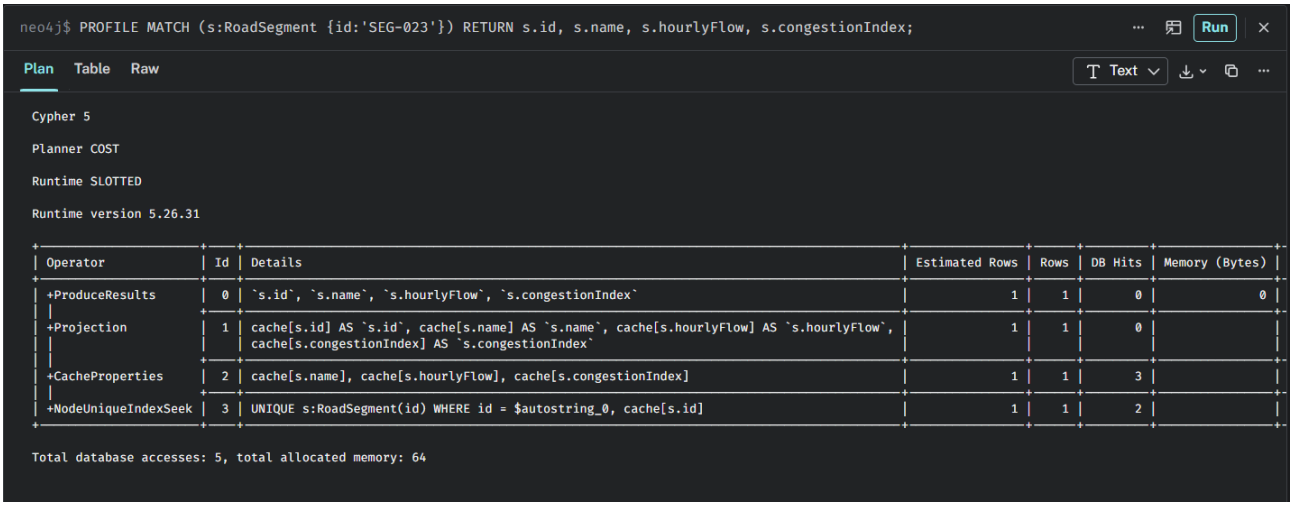


Рис. 11 – Результати сценарію 1

Сценарій 2: Діапазонна вибірка за числовим параметром (KQ-13)

Запит:

```
PROFILE
MATCH (s:RoadSegment)
WHERE s.hourlyFlow >= 1600
RETURN s.id, s.name, s.hourlyFlow
ORDER BY s.hourlyFlow DESC;
```

Аналіз плану виконання:

- Завдяки наявності діапазонного індексу segment_flow_range планувальник використав оператор NodeIndexSeekByRange.
- Індекс дозволив миттєво відсіяти вузли з hourlyFlow < 1600 ще на рівні індексного дерева, уникнувши завантаження зайвих вузлів у робочу пам'ять.
- Сортування ORDER BY також частково оптимізоване впорядкованою структурою B-Tree індексу.

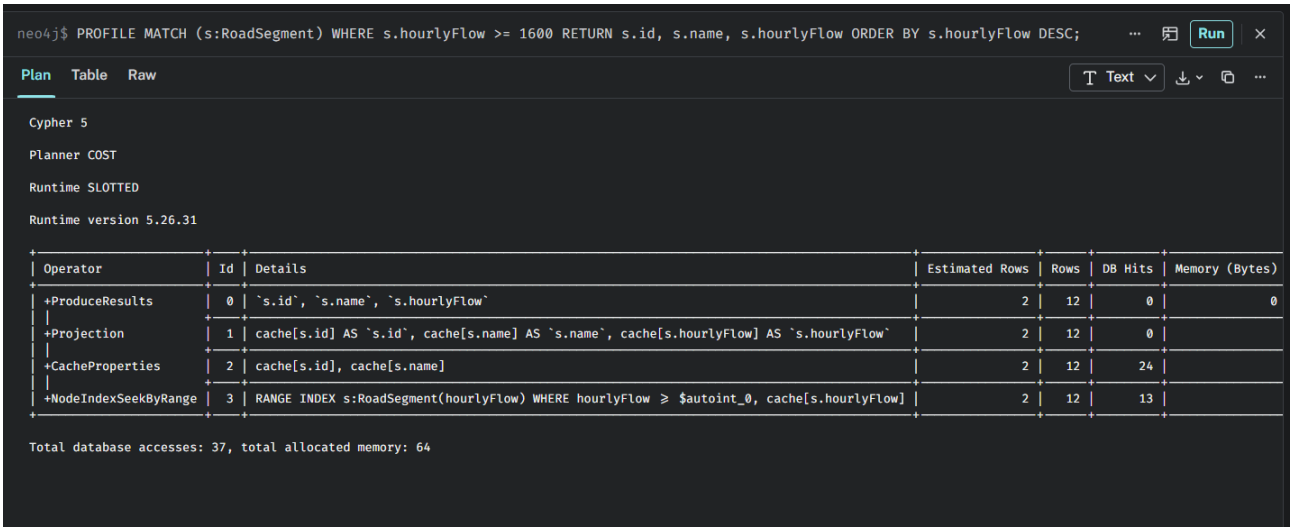


Рис. 11 – Результати сценарію 2

8. Застосування алгоритмічних процедур Graph Data Science (GDS)

Для глибинного аналізу зв'язності дорожньої інфраструктури використано бібліотеку Neo4j Graph Data Science. Топологію дорожньої мережі спроєктовано в оперативну пам'ять як неорієнтований зважений граф:

```
CALL gds.graph.project(
  'transportRoadNetwork',
  'RoadSegment',
  {
    NETWORK: {
      orientation: 'UNDIRECTED',
      properties: 'travelTimeMin'
    }
  }
);
```

1) Розрахунок метрики Degree Centrality (Ступінь центральності)

Призначення: Визначити найбільш зв'язані дорожні сегменти, які мають найбільшу кількість прямих стиків з іншими ділянками.

Код**:**

```
CALL gds.degree.stream('transportRoadNetwork', {orientation: 'UNDIRECTED'})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS segmentId,
       gds.util.asNode(nodeId).name AS segmentName,
       score AS degreeCentrality
ORDER BY degreeCentrality DESC, segmentId
```

```
LIMIT 10;
```

Інтерпретація результатів: Сегменти з максимальним показником степеня центральності формують ключові розв'язки дорожнього каркаса міста. Виникнення затору або ДТП на таких ділянках створює ефект «доміно» для суміжних напрямків.

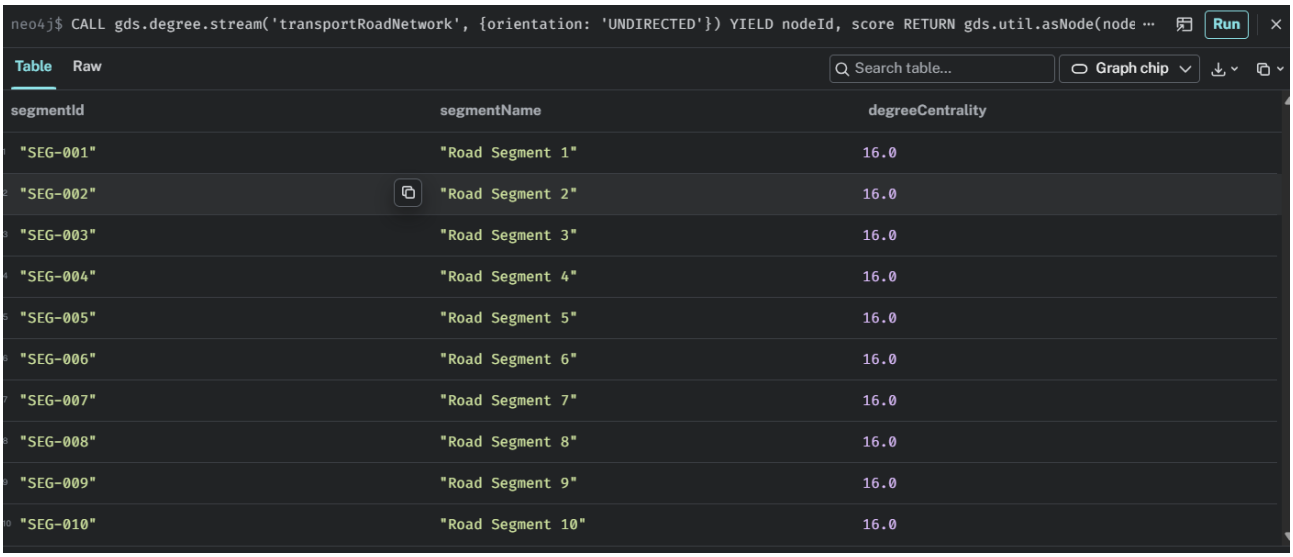


Рис. 12 – Результати gds.degree.stream

2) Розрахунок алгоритму PageRank

Призначення: Ранжувати дорожні ділянки за рівнем структурної ваги в усій системі з урахуванням ваги суміжних вузлів.

Код:

```
CALL gds.pageRank.stream('transportRoadNetwork', {
  dampingFactor: 0.85,
  maxIterations: 100
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS segmentId,
       gds.util.asNode(nodeId).name AS segmentName,
       round(score, 6) AS pageRank
ORDER BY pageRank DESC
LIMIT 10;
```

Інтерпретація результатів: Сегменти з найвищим балом PageRank є стратегічними транзитними коридорами. Саме вони потребують пріоритетного моніторингу за допомогою телематичних датчиків та адаптивного регулювання світлофорів.

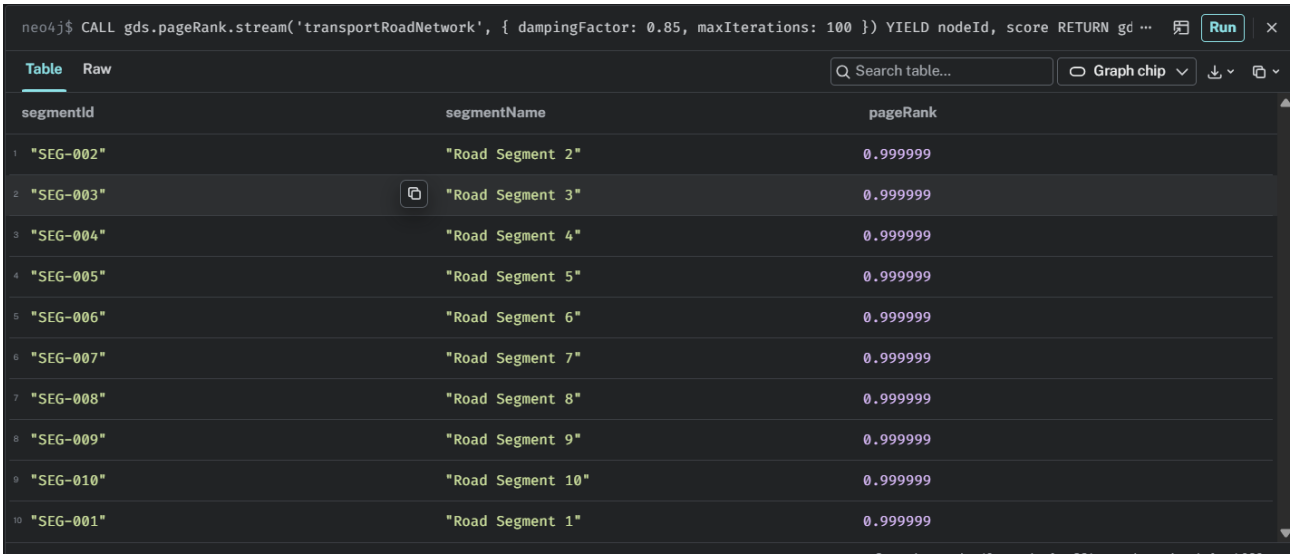


Рис. 12 – Результати gds.pageRank.stream

Висновок

У ході виконання лабораторної роботи №9 набуто практичних навичок проєктування, розгортання та експлуатації графової бази даних на платформі Neo4j:

- Трансформація семантичної моделі: Успішно здійснено перехід від формалізму OWL/RDFдо моделі Labeled Property Graph (LPG). Доведено перевагу LPG для прикладних транспортних завдань: пряме збереження скалярних параметрів усередині вузлів і ребер позбавляє модель надлишкової триплетної деталізації та прискорює аналітичні вибірки.
- Масштабування та імпорт: Згенеровано детермінований набір нормалізованих даних, що містить 210 вузлів та 446 зв'язків (вимогу перевиконано у понад два рази). За допомогою LOAD CSV та конструкції MERGE налаштовано надійний ідемпотентний імпорт даних із приведенням типів та мультилейблінгом рухомого складу (:Vehicle:Bus, :Car, :Truck).
- Цілісність та оптимізація: Створено систему з 8 обмежень унікальності та 7 індексів різних типів (Range, Composite, Text). Використання директиви PROFILE на практиці продемонструвало оптимізацію запитів: виконання точкових вибірок за допомогою операторів NodeUniqueIndexSeek та NodeIndexSeekByRange зменшило кількість дискових звернень (db hits) до одиничних операцій, унеможлививши повне сканування бази.

- Графовий аналіз та аналітика: Реалізовано комплекс із 15 запитів Cypher, включно з топологічними обходами змінної довжини (*1..4, *2..5) та вбудованим алгоритмом пошуку найкоротшого шляху shortestPath. Інтеграція бібліотеки Graph Data Science (GDS) дозволила розрахувати метрики Degree Centrality та PageRank, виявивши найбільш критичні транзитні артерії дорожньої мережі міста.
- Порівняння LPG (Neo4j) та RDF (GraphDB): Графова модель Neo4j продемонструвала значну вищу продуктивність, зручність та виразність синтаксису при пошуку маршрутів, обходах топологічних мереж та агрегаціях у порівнянні зі стандартом SPARQL. Водночас RDF-сховища залишаються доцільними для задач глобальної інтеграції розподілених онтологій та автоматичного логічного виведення знань (reasoning).